# FourSight Intent Classification Challenge

## Background

FourKites' assistant, **FourSight**, is a chat interface in front of several different products. The first decision it must make for any question is **where the question belongs** — because the answer lives in a different system depending on what is asked. The seven destinations:

| Intent | What it answers |
|---|---|
| `shipment_information` | Shipment Visibility — "where is my shipment?" (current location / visibility state). |
| `realtime_query` | ETAs, statuses and reference-number lookups for specific loads (live operational data). |
| `analytics` | Aggregated answers from reasoning over historic data: counts, trends, breakdowns. |
| `disruptions` | Disruptions affecting shipments or the supply chain: weather, ports, borders, capacity. |
| `chitchat` | Generic conversation with the bot: "hi", "how are you", "what can you do". |
| `knowledge_base` | Understanding a product or feature: "how do I add a user", "how often does it update". |
| `other` | Out of FourKites' scope, or fits none of the classes above. |

**Why this separation matters.** The data behind these answers is spread across multiple databases and products. There is no single backend that serves all of them, so there is no unified solution — the assistant must route each question to the right product first. A wrong route means the right system is never queried. This first-hop intent classification is the foundation the whole assistant stands on.

Your task: read a user's question and return its intent — one of the seven classes above — **deterministically** (same input always yields the same label).

## The task

- This is **not** a "call an LLM and hope" exercise. A fine-tuned encoder (BERT / DistilBERT / RoBERTa / DeBERTa), a classifier over embeddings, or any approach that fits the use case is encouraged. A generative model is fine only if it emits a deterministic, constrained label.
- Inference must be **deterministic**: same question in, same label out, every time.
- The seven labels are a **closed set**. Every prediction must be one of them.

## This is a chat system — intent depends on conversational history

> In production FourSight is a **multi-turn chat**, and the intent of the current question often depends on what was said earlier (e.g. "what about last month?" is `analytics` only because of the turn before it). **The dataset shared with you is single-turn** — one question and its label per row, with no conversation context.
>
> Your report **must** address how your approach would incorporate conversational history, and **you will be evaluated on this**: the design for the multi-turn setting, not only single-turn accuracy. An accurate single-turn model with a credible, concrete story for using history is worth far more than one that ignores it.

## The data

```
data/
  train.csv    1,904 labelled questions   (id, question, intent)
```

```
test.csv       566 questions           (id, question)  - no labels; we score these
```

- Each row is one real user question, **anonymised** — all personal and company identifiers removed.
- `train.csv` has the gold intent; `test.csv` does not — you predict it and submit.
- **No separate dev split.** Hold out your own stratified validation set from `train.csv` and use `score.py` to self-check.
- Classes are **imbalanced** (see `labels.json`). Rare classes (`chitchat`, `disruptions`, `other`) count as much as common ones in scoring.
- The **shipment_information / realtime_query / analytics** boundary is deliberately hard; that is where good solutions separate from memorised ones.

## Rules

- Predict exactly **one** of the seven labels for every test row. Every row counts.
- A label outside the seven-label set, or a missing row, scores as **wrong**.
- Don't label the test rows by hand; don't try to de-anonymise the data.

## Scoring

```
python3 score.py --gold data/test.csv --pred your_predictions.csv
```

`score.py` is the exact scorer we run on the hidden test labels. Because the classes are imbalanced, the headline number weights the macro average heavily:

```
OVERALL = 0.6 * macro-F1   (every class weighted equally; rewards rare-class handling)
        + 0.4 * accuracy    (overall correctness)
```

We also report **per-class precision / recall / F1**, the confusion matrix, and the most-confused pairs — we look closely at the hard three-way boundary and the rare classes. A classifier that is 90% right and **knows which questions it is unsure of** (a calibrated “needs review” / abstention signal) is worth more than one that is 93% right and can't tell.

## What to submit

- `test_predictions.csv` — predictions for all 566 test rows, in the format of `sample_submission.csv` (`id,intent`).
- `report.pdf` (about 2 pages): your approach and why (and why it's deterministic); validation results — overall, **per-class**, and the confusion matrix, and where it fails; **how you would incorporate conversational history** (required); how you'd handle a brand-new intent in production; **(optional)** a per-prediction confidence / “needs review” flag with its precision and recall; and the GitHub, Hugging Face and W&B; links so we can reproduce it.
- A **GitHub repository** with all code and instructions to reproduce predictions from `test.csv`: seeded and documented, re-runnable end to end.
- A **Hugging Face Hub link** to any model you train, with a short model card (training data, intended use, metrics, limitations). If you train nothing, state that; this item is then not required.
- A **Weights & Biases run link** if you train or fine-tune anything: training and validation loss curves, dev `OVERALL` and its per-class breakdown, and the run config / hyper-parameters. If nothing is trained, not required.

Make the GitHub repo, Hugging Face model and W&B; project **public**, or share access, and include all links in `report.pdf`.

## Constraints

- Any open-weight model, library or technique — fine-tuning, classical ML on embeddings, encoder models, or distilled small LMs.
- Inference must run on a **single GPU with 24 GB or less, or on CPU**. **No paid hosted APIs at inference time.** Free Colab / Kaggle GPUs are fine for training and evaluation.
- Suggested time: about **6–8 hours**. We value a clear, honest analysis over a leaderboard number.

## What we look for

- correctness on the hard boundaries, not just memorisable questions;
- honest handling of uncertainty (calibration, abstention, “needs review”);
- a concrete, credible design for the multi-turn / conversational-history setting;
- clean, reproducible, deterministic code; clear reasoning about failure modes and trade-offs.

CONFIDENTIAL · © 2026 FourKites, Inc. · Proprietary. For the named recipient's interview evaluation only; do not distribute.